High Availability IoT Telemetry System

Distributed NoSQL Cluster over LXD Infrastructure

Developed by:
Jaime Hernández Pérez

Made towards the modules:
Virtualization Systems (Prof. Fazio)
NoSQL Databases (Prof. Ruggeri)

Contents

1 Introduction **2**

2 Environment and Virtualization **2**
2.1 Hypervisor Setup 2
2.2 LXD Installation and Initialization 3
2.3 Remote Node Clustering 3

3 Persistent Storage **4**

4 Network Interconnectivity **4**

5 Experimental Live Migration vs. Replica Sets **5**
5.1 The CRIU Experiment 5

6 MongoDB Cluster Deployment **5**
6.1 Installation and Initialization 5

7 Real-Time IoT Telemetry Pipeline **6**

8 Disaster Recovery Testing **7**

9 Conclusion **8**

1 Introduction

This report details the design, implementation, and testing of a distributed IoT telemetry system. The architecture fulfills dual requirements: establishing a robust, fault-tolerant infrastructure using virtualization and network routing, and implementing a NoSQL database pipeline.

2 Environment and Virtualization

The foundation relies on Type-2 virtualization running Linux containers to isolate workloads while maximizing performance.

2.1 Hypervisor Setup

Two Virtual Machines were created using VirtualBox to simulate physical distinct nodes:

- **Operating System:** Debian 12.9.0 (amd64-netinst).
- **Resources:** 2 CPU Cores and 2048 MB RAM per node.
- **Networking:** Bridged Adapters to allow direct L2 communication.

2.2 LXD Installation and Initialization

LXD was installed through the Snap package manager. The initialization was done using the automated flag, which configures a default loop-backed storage pool and an lxdbr0 network bridge:

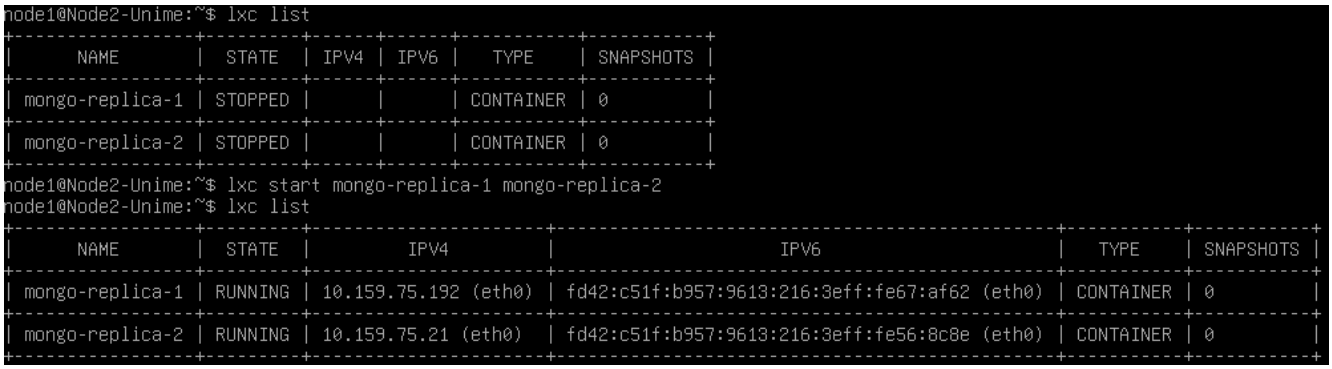
```
sudo apt update && sudo apt upgrade -y
sudo snap install core
sudo snap install lxd
sudo usermod -aG lxd $USER
lxd init --auto
```

2.3 Remote Node Clustering

To allow management across hosts, Node 1 was configured to accept remote LXD connections on port 8443, and Node 2 was joined as a client using a trust password:

```
# On Node 1
lxc config set core.https_address [::]:8443
lxc config set core.trust_password rootme

# On Node 2
lxc remote add node1 192.168.190.215
```



```
node1@Node2-Unime:~$ lxc list
+-----------------+---------+------+------+-----------+-----------+
|      NAME       |  STATE  | IPV4 | IPV6 |   TYPE    | SNAPSHOTS |
+-----------------+---------+------+------+-----------+-----------+
| mongo-replica-1 | STOPPED |      |      | CONTAINER | 0         |
+-----------------+---------+------+------+-----------+-----------+
| mongo-replica-2 | STOPPED |      |      | CONTAINER | 0         |
+-----------------+---------+------+------+-----------+-----------+
node1@Node2-Unime:~$ lxc start mongo-replica-1 mongo-replica-2
node1@Node2-Unime:~$ lxc list
+-----------------+---------+----------------------+-----------------------------------------------+-----------+-----------+
|      NAME       |  STATE  |         IPV4         |                      IPV6                     |   TYPE    | SNAPSHOTS |
+-----------------+---------+----------------------+-----------------------------------------------+-----------+-----------+
| mongo-replica-1 | RUNNING | 10.159.75.192 (eth0) | fd42:c51f:b957:9613:216:3eff:fe67:af62 (eth0) | CONTAINER | 0         |
+-----------------+---------+----------------------+-----------------------------------------------+-----------+-----------+
| mongo-replica-2 | RUNNING | 10.159.75.21 (eth0)  | fd42:c51f:b957:9613:216:3eff:fe56:8c8e (eth0) | CONTAINER | 0         |
+-----------------+---------+----------------------+-----------------------------------------------+-----------+-----------+
```

Figure 1: LXD container list on Node 2 showing active IPv4 assignments.

3 Persistent Storage

To prevent data loss during container failures, a Network File System (NFS) was established. Node 1 acts as the storage server, exporting the directory to Node 2:

```
# On Node 1 (Server)
sudo apt install nfs-kernel-server -y
sudo mkdir -p /mnt/lxd_pool
sudo chown nobody:nogroup /mnt/lxd_pool
sudo chmod 777 /mnt/lxd_pool
echo "/mnt/lxd_pool 192.168.190.0/24(rw,sync,no_subtree_check)" |
    sudo tee -a /etc/exports
sudo exportfs -a
sudo systemctl restart nfs-kernel-server

# On Node 2 (Client)
sudo apt install nfs-common -y
sudo mount -t nfs 192.168.190.215:/mnt/lxd_pool /mnt/lxd_pool
```

Three containers were launched: mongo-primary, mongo-replica-1, mongo-replica-2. The NFS share was mapped directly into their internal database paths:

```
lxc config device add mongo-primary nfs-disk disk source=/mnt/
   lxd_pool/mongo-primary path=/data/db
```

4 Network Interconnectivity

Containers on Node 1 (10.37.37.0/24) could not communicate with containers on Node 2 (10.159.75.0/24) due to bridge isolation. Static IP routing was implemented at the host level:

```
# On Node 1 (Routing traffic via Node 2)
sudo ip route add 10.159.75.0/24 via 192.168.190.214

# On Node 2 (Routing traffic via Node 1)
sudo ip route add 10.37.37.0/24 via 192.168.190.215
```

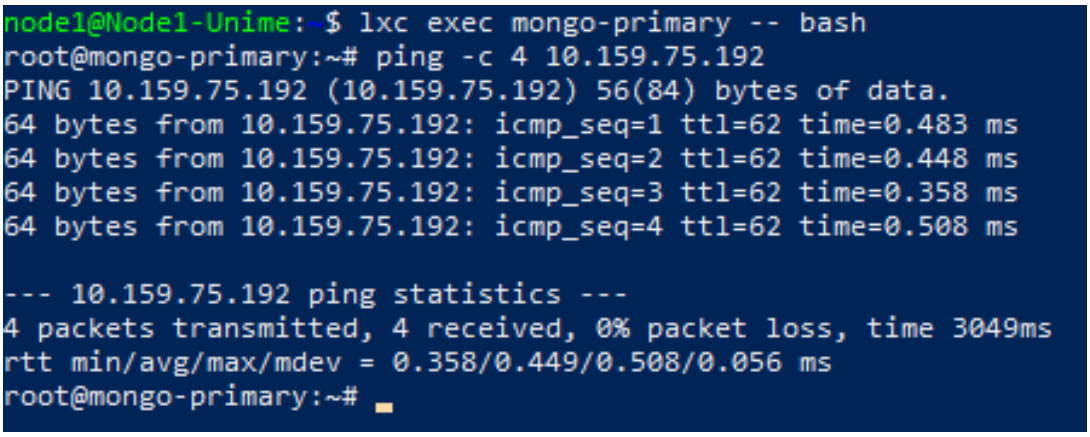


Figure 2: Successful ICMP ping across the statically routed LXD subnets.

5 Experimental Live Migration vs. Replica Sets

5.1 The CRIU Experiment

Initially, I explored live migration to achieve High Availability using CRIU (Check Point/Restore In Userspace). A test container (db-prod) was created, and CRIU was enabled in LXD:

```
sudo apt install snapd criu -y
sudo snap set lxd criu.enable=true
lxc move node1:db-prod local:db-prod
```

I monitored state using a heartbeat_log.txt file. However, migrating an active database container across the network caused severe lock issues and network timeouts. Due to this instability, CRIU was abandoned (lxc delete db-prod –force) in favor of native application-layer redundancy.

6 MongoDB Cluster Deployment

MongoDB 7.0 was deployed to form a distributed Replica Set, natively handling failovers without hypervisor interference.

6.1 Installation and Initialization

To bypass terminal hanging issues caused by the needrestart utility in Ubuntu, a non-interactive installation was forced:

```
DEBIAN_FRONTEND=noninteractive apt-get install -y mongodb-org
```

The mongod.conf was modified to bind to 0.0.0.0 and join the rs0 replica set. The primary node initiated the cluster:

```
rs.initiate({
  _id: "rs0",
  members: [
    { _id: 0, host: "10.37.37.29:27017" },
    { _id: 1, host: "10.159.75.192:27017" },
    { _id: 2, host: "10.159.75.21:27017" }
  ]
})
```

7 Real-Time IoT Telemetry Pipeline

A data ingestion pipeline was constructed using Python and MQTT (Mosquitto).

```
root@mongo-primary:~# python3 sensor.py
Sensor simulator running. Press Ctrl+C to stop.
[Sensor] Published: {'sensor_id': 'station_01', 'temp': 22.95, 'hum': 47.96, 'timestamp': 1777308227.8707705}
[Sensor] Published: {'sensor_id': 'station_01', 'temp': 20.45, 'hum': 44.8, 'timestamp': 1777308229.8733337}
[Sensor] Published: {'sensor_id': 'station_01', 'temp': 23.53, 'hum': 47.82, 'timestamp': 1777308231.8758643}
[Sensor] Published: {'sensor_id': 'station_01', 'temp': 22.94, 'hum': 45.0, 'timestamp': 1777308233.8781726}
[Sensor] Published: {'sensor_id': 'station_01', 'temp': 24.13, 'hum': 42.13, 'timestamp': 1777308235.8788397}
[Sensor] Published: {'sensor_id': 'station_01', 'temp': 20.8, 'hum': 47.35, 'timestamp': 1777308237.8813274}
```

Figure 3: Python IoT simulator publishing telemetry data to the Mosquitto MQTT broker.

1. **Sensor Simulator (sensor.py):** Acts as an IoT edge device, publishing random JSON payloads to the MQTT broker.
2. **Data Ingestor (ingest.py):** A background daemon that subscribes to MQTT and inserts the data into the iot_telemetry MongoDB database.

```
root@mongo-primary:~# mongosh
Current Mongosh Log ID: 69ef926ab928ccba8d44ba88
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.2
Using MongoDB:          7.0.31
Using Mongosh:          2.8.2

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-04-27T16:20:16.665+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http:/
   2026-04-27T16:20:17.999+00:00: Access control is not enabled for the database. Read and write access to data and configuration
   2026-04-27T16:20:17.999+00:00: For customers running MongoDB 7.0, we suggest changing the contents of the following sysfsFile
------

rs0 [direct: secondary] test> use iot_telemetry
switched to db iot_telemetry
rs0 [direct: secondary] iot_telemetry> db.sensors.find().sort({timestamp: -1}).limit(3)
[
  {
    _id: ObjectId('69e947b98dca1864ec9c22dc'),
    sensor_id: 'station_01',
    temp: 24.56,
    hum: 44.23,
    timestamp: 1776895929.588529
  },
  {
    _id: ObjectId('69e947b78dca1864ec9c22db'),
    sensor_id: 'station_01',
    temp: 22.87,
    hum: 47.98,
    timestamp: 1776895927.585795
  },
  {
    _id: ObjectId('69e947b58dca1864ec9c22da'),
    sensor_id: 'station_01',
    temp: 23.37,
    hum: 41.02,
    timestamp: 1776895925.5833733
  }
]
```

Figure 4: JSON payload retrieval from the MongoDB telemetry collection.

8 Disaster Recovery Testing

To prove system resilience, a hardware failure was simulated by forcing a crash on the primary container:

```
lxc stop mongo-primary --force
```

By executing rs.status() on Node 2, we observed the cluster's autonomous recovery. The secondary node automatically won the election and was promoted to PRIMARY.


```
members: [
  {
    _id: 0,
    name: '10.37.37.29:27017',
    health: 0,
    state: 8,
    stateStr: '(not reachable/healthy)',
    uptime: 0,
    optime: { ts: Timestamp({ t: 0, i: 0 }), t: Long('-1') },
    optimeDurable: { ts: Timestamp({ t: 0, i: 0 }), t: Long('-1') },
    optimeDate: ISODate('1970-01-01T00:00:00.000Z'),
    optimeDurableDate: ISODate('1970-01-01T00:00:00.000Z'),
    lastAppliedWallTime: ISODate('2026-04-27T16:45:50.267Z'),
    lastDurableWallTime: ISODate('2026-04-27T16:45:50.267Z'),
    lastHeartbeat: ISODate('2026-04-27T16:47:15.716Z'),
    lastHeartbeatRecv: ISODate('2026-04-27T16:45:51.681Z'),
    pingMs: Long('0'),
    lastHeartbeatMessage: "Couldn't get a connection within the time limit",
    syncSourceHost: '',
    syncSourceId: -1,
    infoMessage: '',
    configVersion: 1,
    configTerm: 6
  },
  {
    _id: 1,
    name: '10.159.75.192:27017',
    health: 1,
    state: 1,
    stateStr: 'PRIMARY',
    uptime: 1653,
    optime: { ts: Timestamp({ t: 1777308440, i: 1 }), t: Long('6') },
    optimeDate: ISODate('2026-04-27T16:47:20.000Z'),
    lastAppliedWallTime: ISODate('2026-04-27T16:47:20.272Z'),
    lastDurableWallTime: ISODate('2026-04-27T16:47:20.272Z'),
    syncSourceHost: '',
    syncSourceId: -1,
    infoMessage: '',
    electionTime: Timestamp({ t: 1777306797, i: 1 }),
    electionDate: ISODate('2026-04-27T16:19:57.000Z'),
    configVersion: 1,
    configTerm: 6,
    self: true,
    lastHeartbeatMessage: ''
  },
  {
    _id: 2,
    name: '10.159.75.21:27017',
    health: 1,
    state: 2,
    stateStr: 'SECONDARY',
    uptime: 1648,
    optime: { ts: Timestamp({ t: 1777308430, i: 1 }), t: Long('6') },
    optimeDurable: { ts: Timestamp({ t: 1777308430, i: 1 }), t: Long('6') },
    optimeDate: ISODate('2026-04-27T16:47:10.000Z'),
    optimeDurableDate: ISODate('2026-04-27T16:47:10.000Z'),
    lastAppliedWallTime: ISODate('2026-04-27T16:47:20.272Z'),
    lastDurableWallTime: ISODate('2026-04-27T16:47:20.272Z'),
    lastHeartbeat: ISODate('2026-04-27T16:47:19.704Z'),
    lastHeartbeatRecv: ISODate('2026-04-27T16:47:20.526Z'),
    pingMs: Long('0'),
    lastHeartbeatMessage: '',
    syncSourceHost: '10.159.75.192:27017',
    syncSourceId: 1,
    infoMessage: '',
    configVersion: 1,
    configTerm: 6
  }
],
```


Figure 5: Replica Set status after a simulated crash, showing the automated promotion of a secondary node.

Querying the new primary returned the exact same JSON telemetry payloads generated prior to the crash. Because the database files were continuously written to the NFS shared storage, the destruction of the compute container had zero impact on the persistent data layer.

9 Conclusion

The project successfully integrates a distributed network infrastructure with a modern NoSQL telemetry pipeline. The experimentation phase proved that native Replica Sets combined with shared NFS storage provide a vastly superior High Availability architecture compared to hypervisor-level CRIU migrations. The system fulfills all requirements for scalability, persistence, and fault tolerance.

References

[1] Hernández Pérez, Jaime. (2026). *High Availability IoT Telemetry System - Source Code Repository*. GitHub. `https://github.com/JaimeHP05/High-Availability-IoT-Telemetry-System`

[2] Debian Project. (2024). *Debian GNU/Linux Installation Guide*. [Online]. Available: `https://www.debian.org/releases/stable/installmanual`

[3] Canonical Ltd. (2024). *LXD Documentation: Introduction to Linux Containers*. [Online]. Available: `https://documentation.ubuntu.com/lxd/en/latest/`

[4] Fazio, M. *Learning Materials: Virtualization Systems*. [E-learning platform]. University of Messina / UNIME.

[5] IETF. (2003). *RFC 3530: Network File System (NFS) version 4 Protocol*. [Online]. Available: `https://tools.ietf.org/html/rfc3530`

[6] MongoDB Inc. (2024). *MongoDB Manual: Replication and High Availability*. [Online]. Available: `https://www.mongodb.com/docs/manual/replication/`

[7] Ruggeri, A. *Learning Materials: NoSQL Databases*. [E-learning platform]. University of Messina / UNIME.

[8] OASIS Standard. (2014). *MQTT Version 3.1.1 Specification*. [Online]. Available: `https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html`

[9] Eclipse Mosquitto. *An Open Source MQTT Broker Documentation*. [Online]. Available: `https://mosquitto.org/documentation/`